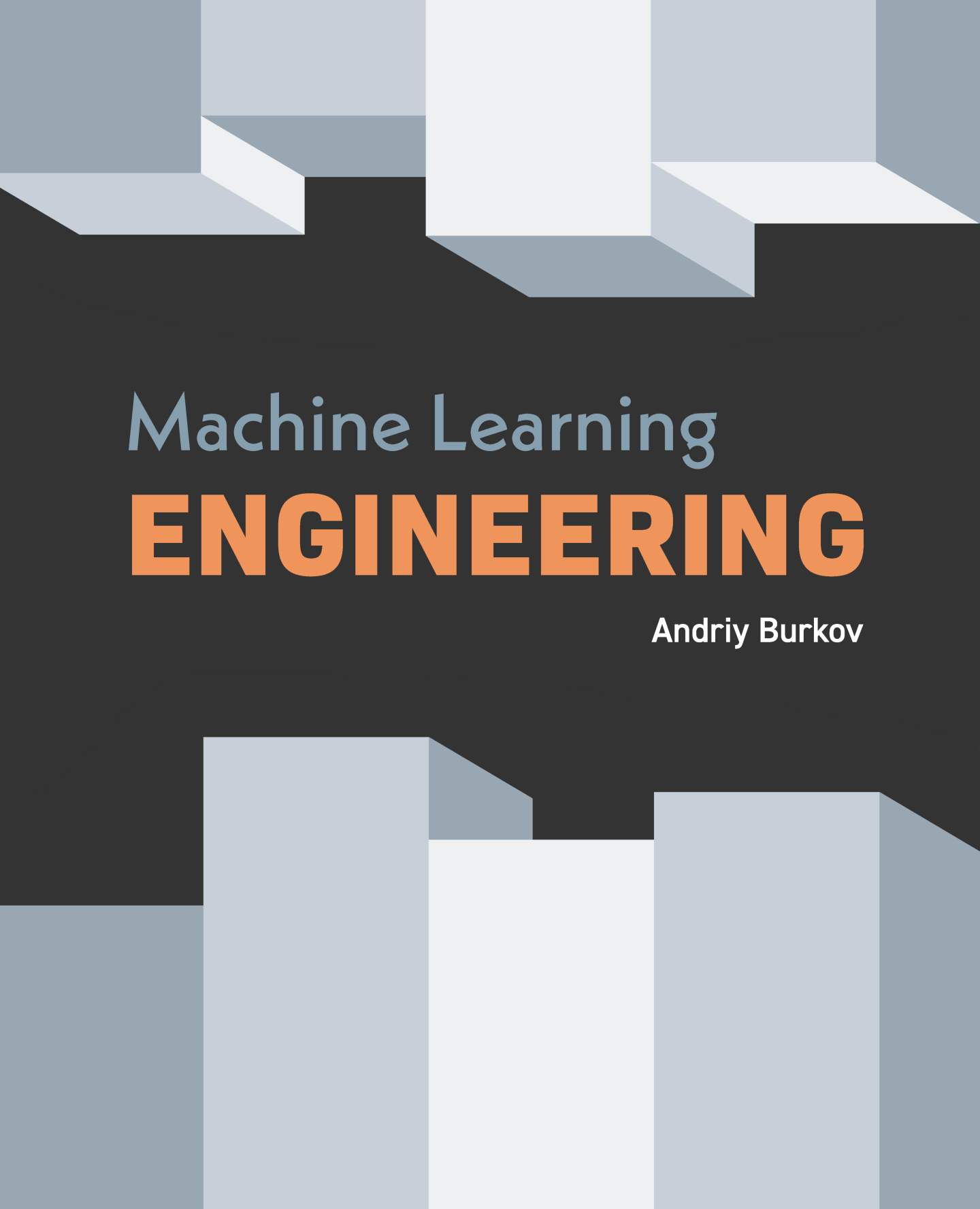

Machine Learning
ENGINEERING
Andriy Burkov

“In theory, there is no difference between theory and practice. But in practice, there is.”

— Benjamin Brewster

“The perfect project plan is possible if one first documents a list of all the unknowns.”

— Bill Langley

“When you’re fundraising, it’s AI. When you’re hiring, it’s ML. When you’re implementing, it’s linear regression. When you’re debugging, it’s printf().”

— Baron Schwartz

The book is distributed on the “read first, buy later” principle.

Foreword

Foreword by **Cassie Kozyrkov**, Chief Decision Scientist at Google, author of the course ***Making Friends with Machine Learning*** on Google Cloud Platform

I'd like to let you in on a secret: when people say "machine learning" it sounds like there's only one discipline here. Surprise! There are actually two machine learnings, and they are as different as innovating in food recipes and inventing new kitchen appliances. Both are noble callings, as long as you don't get them confused; imagine hiring a pastry chef to build you an oven or an electrical engineer to bake bread for you!

The bad news is that almost everyone does mix these two machine learnings up. No wonder so many businesses fail at machine learning as a result. What no one seems to tell beginners is that most machine learning courses and textbooks are about Machine Learning Research — how to build ovens (and microwaves, blenders, toasters, kettles... the kitchen sink!) from scratch, not how to cook things and innovate with recipes at enormous scale. In other words, if you're looking for opportunities to create innovative ML-based solutions to business problems, you want the discipline called Applied Machine Learning, not Machine Learning Research, so most books won't suit your needs.

And now for the good news! You're looking at one of the few true Applied Machine Learning books out there. That's right, you found one! A real applied needle in the haystack of research-oriented stuff. Excellent job, dear reader... unless what you were actually looking for is a book to help you learn the skills to design general purpose algorithms, in which case I hope the author won't be too upset with me for telling you to flee now and go pick up pretty much any other machine learning book. This one is different.

When I created Making Friends with Machine Learning in 2016, Google's Applied Machine Learning course loved by more than ten thousand of our engineers and leaders, I gave it a very similar structure to the one in this book. That's because doing things in the right order is crucial in the applied space. As you use your newfound data powers, tackling certain steps before you've completed others can lead to anything from wasted effort to a project-demolishing kablooie. In fact, the similarity in table of contents between this book and my course is what originally convinced me to give this book a read. In a clear case of convergent evolution, I saw in the author a fellow thinker kept up at night by the lack of available resources on Applied Machine Learning, one of the most potentially-useful yet horribly-misunderstood areas of engineering, enough to want to do something about it. So, if you're about to close this book, how about you do me a quick favor and at least ponder why the Table of Contents is arranged the way it is. You'll learn something good just from that, I promise.

So, what's in the rest of the book? The machine learning equivalent of a bumper guide to innovating in recipes to make food at scale. Since you haven't read the book yet, I'll put

it in culinary terms: you'll need to figure out what's worth cooking / what the objectives are (*decision-making and product management*), understand the suppliers and the customers (*domain expertise and business acumen*), how to process ingredients at scale (*data engineering and analysis*), how to try many different ingredient-appliance combinations quickly to generate potential recipes (*prototype phase ML engineering*), how to check that the quality of the recipe is good enough to serve (*statistics*), how to turn a potential recipe into millions of dishes served efficiently (*production phase ML engineering*), and how to ensure that your dishes stay top notch even if the delivery truck brings you a ton of potatoes instead of the rice you ordered (*reliability engineering*). This book is one of the few to offer perspectives on each step of the end-to-end process.

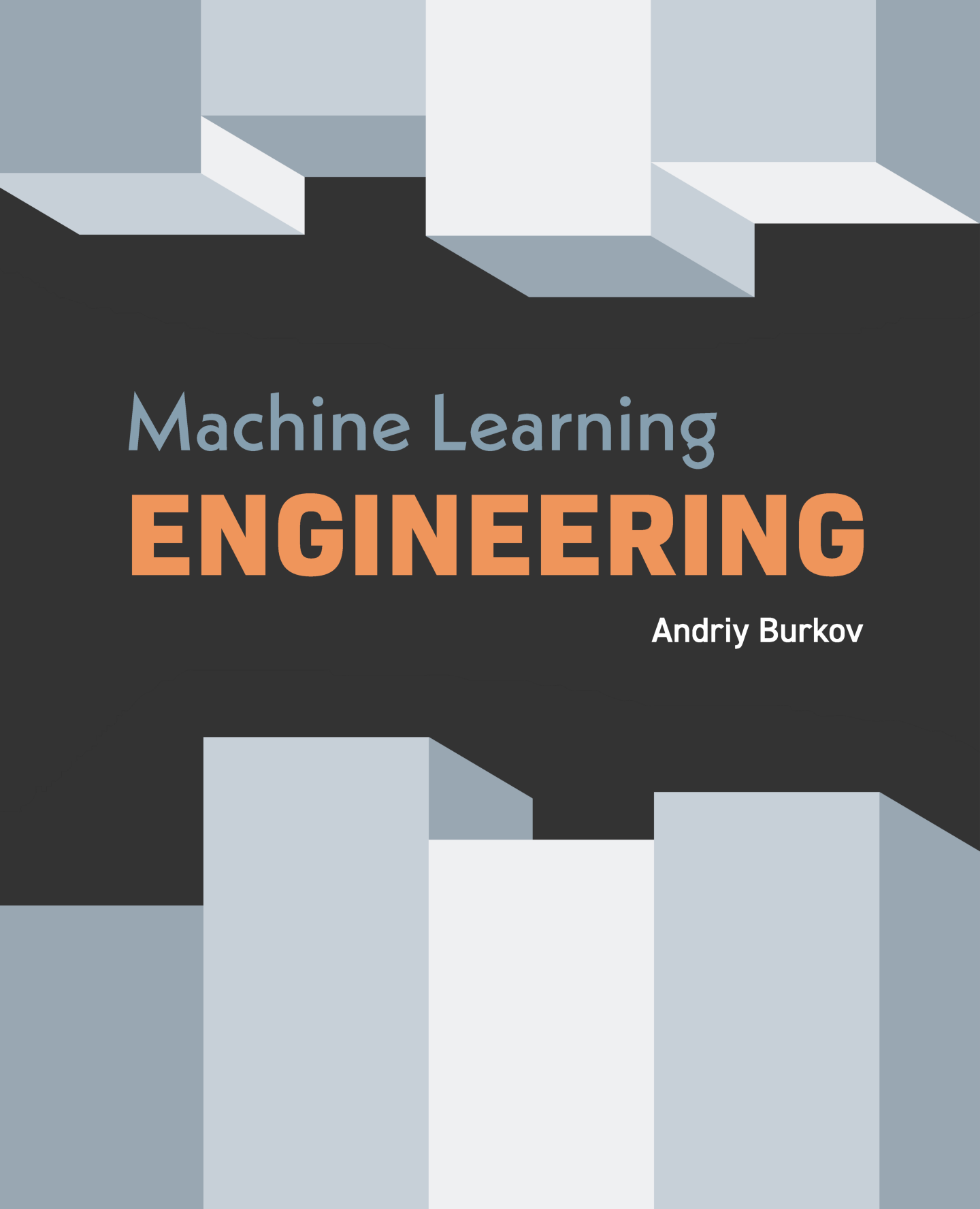
Now would be a good moment for me to be blunt with you, dear reader. This book *is* pretty good. It is. Really. But it's not perfect. It cuts corners on occasion — just like a professional machine learning engineer is wont to do — though on the whole it gets its message right. And, since it covers an area with rapidly-evolving best practices, it doesn't pretend to offer the last word on the subject. But even if it were terribly sloppy, it would still be worth reading. Given how few comprehensive guides to Applied Machine Learning are out there, a coherent introduction to these topics is worth its weight in gold. I'm so glad this one is here!

One of my favorite things about this book is how fully it embraces the most important thing you need to know about machine learning: mistakes are possible... and sometimes they hurt. As my colleagues in site reliability engineering love to say, "Hope is not a strategy." Hoping that there will be no mistakes is the worst approach you can take. This book does so much better. It promptly shatters any false sense of security you were tempted to have about building an AI system that is more "intelligent" than you are. (Um, no. Just no.) Then it diligently takes you through a survey of all kinds of things that can go wrong in practice and how to prevent/detect/handle them. This book does a great job of outlining the importance of monitoring, how to approach model maintenance, what to do when things go wrong, how to think about fallback strategies for the kinds of mistakes you can't anticipate, how to deal with adversaries who try to exploit your system, and how to manage the expectations of your human users (there's also a section on what to do when your, er, users are machines). These are hugely important topics in practical machine learning, but they're so often neglected in other books. Not here.

If you intend to use machine learning to solve business problems at scale, I'm delighted you got your hands on this book. Enjoy!

Cassie Kozyrkov

September 2020


Machine Learning
ENGINEERING
Andriy Burkov

“In theory, there is no difference between theory and practice. But in practice, there is.”

— Benjamin Brewster

“The perfect project plan is possible if one first documents a list of all the unknowns.”

— Bill Langley

“When you’re fundraising, it’s AI. When you’re hiring, it’s ML. When you’re implementing, it’s linear regression. When you’re debugging, it’s printf().”

— Baron Schwartz

The book is distributed on the “read first, buy later” principle.

Preface

During the past several years, machine learning (ML), for many, has become a synonym for artificial intelligence. Even though machine learning, as a field of science, has existed for several decades, only a handful of organizations in the world have fully harnessed its potential. Despite the availability of modern open-source machine learning libraries, packages and frameworks supported by the leading organizations and broad communities of scientists and software engineers, most organizations are still struggling to apply machine learning for solving practical business problems.

One difficulty lies in the scarcity of talent. However, even when they have access to talented machine learning engineers and data analysts, in 2020, most organizations[1] still spend between 31 and 90 days deploying one model, while 18 percent of companies are taking longer than 90 days — some spending more than a year productionizing. The main challenges organizations face when developing ML capabilities, such as model version control, reproducibility, and scaling, are rather engineering than scientific.

There are plenty of good books on machine learning, both theoretical and hands-on. From a typical machine learning book, you can learn the types of machine learning, major families of algorithms, how they work, and how to build models from data using those algorithms.

A typical machine learning book is less concerned with the engineering aspects of implementing machine learning projects. Such questions as data collection, storage, preprocessing, feature engineering, as well as testing and debugging of models, their deployment to and retirement from production, runtime and post-production maintenance, are often left outside the scope of machine learning books.

This book intends to fill that gap.

Who This Book is For

I assume that the reader of this book understands machine learning basics and is capable of building a model, given a properly formatted dataset using a favorite programming language or a machine learning library. If you don't feel comfortable applying machine learning algorithms to data and don't clearly see the difference between logistic regression, support vector machine, and random forest, I recommend starting your journey with The Hundred-Page Machine Learning Book, and then move to this book.

The target audience of this book is data analysts who lean towards a machine learning engineering role, machine learning engineers who want to bring more structure to their work, machine learning engineering students, as well as software architects who happen to deal with models provided by data analysts and machine learning engineers.

[1] "2020 state of enterprise machine learning", Algorithmia, 2019.

How to Use This Book

This book is a comprehensive review of machine learning engineering best practices and design patterns. I recommend reading it from beginning to end. However, you can read chapters in any order as they cover distinct aspects of the machine learning project lifecycle and do not have direct dependencies.

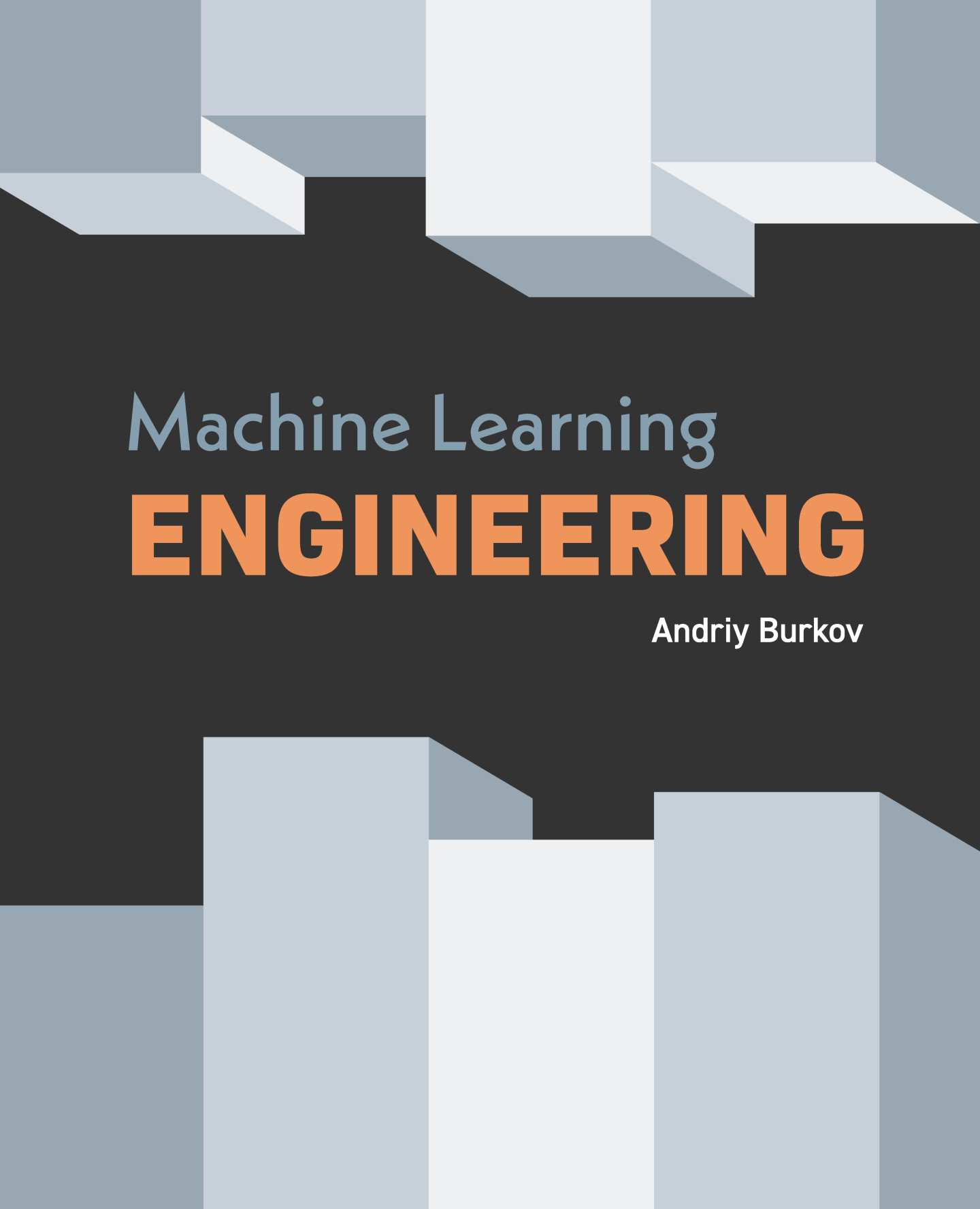
Should You Buy This Book?

Like its companion and precursor The Hundred-Page Machine Learning Book, this book is distributed on the "read-first, buy-later" principle. I firmly believe that readers must be able to read a book before paying for it; otherwise, they buy a pig in a poke.

The "read-first, buy-later" principle implies that you can freely download the book, read it, and share it with your friends and colleagues. If you read and liked the book, or found it helpful or useful in your work, business, or studies, then buy it.

Now you are all set. Enjoy your reading!

Andriy Burkov


Machine Learning
ENGINEERING
Andriy Burkov

“In theory, there is no difference between theory and practice. But in practice, there is.”

— Benjamin Brewster

“The perfect project plan is possible if one first documents a list of all the unknowns.”

— Bill Langley

“When you’re fundraising, it’s AI. When you’re hiring, it’s ML. When you’re implementing, it’s linear regression. When you’re debugging, it’s printf().”

— Baron Schwartz

The book is distributed on the “read first, buy later” principle.

1 Introduction

Though the reader of this book should have a basic understanding of machine learning, it is still important to start with definitions, so that we are sure that we have a common understanding of the terms used throughout the book.

Below, I repeat some of the definitions from Chapter 2 of The Hundred-Page Machine Learning Book and also give several new ones. If you read my first book, some parts of this chapter might sound familiar.

After reading this chapter, we will understand the same way such concepts as supervised and unsupervised learning. We will agree on the data terminology, such as data used directly and indirectly, raw and tidy data, training and holdout data.

We will know when to use machine learning, when not to use it, and various forms of machine learning such as model- and instance-based, deep and shallow, classification and regression, and others.

Finally, we will define the scope of machine learning engineering and introduce the machine learning project lifecycle.

1.1 Notation and Definitions

Let's start by stating the basic mathematical notation and define the terms and notions, to which we will often have recourse in this book.

1.1.1 Data Structures

A **scalar**[1] is a simple numerical value, like 15 or -3.25. Variables or constants that take scalar values are denoted by an italic letter, like x or a.

A **vector** is an ordered list of scalar values, called attributes. We denote a vector as a bold character, for example, $\mathbf{x}$ or $\mathbf{w}$. Vectors can be visualized as arrows that point to some directions as well as points in a multi-dimensional space. Illustrations of three two-dimensional vectors, $\mathbf{a} = [2, 3]$, $\mathbf{b} = [-2, 5]$, and $\mathbf{c} = [1, 0]$ are given in Figure 1. We denote an attribute of a vector as an italic value with an index, like this: $w^{(j)}$ or $x^{(j)}$. The index j denotes a specific **dimension** of the vector, the position of an attribute in the list. For instance, in the vector $\mathbf{a}$ shown in red in Figure 1, $a^{(1)} = 2$ and $a^{(2)} = 3$.

[1] If a term is **in bold**, that means that the term can be found in the index at the end of the book.

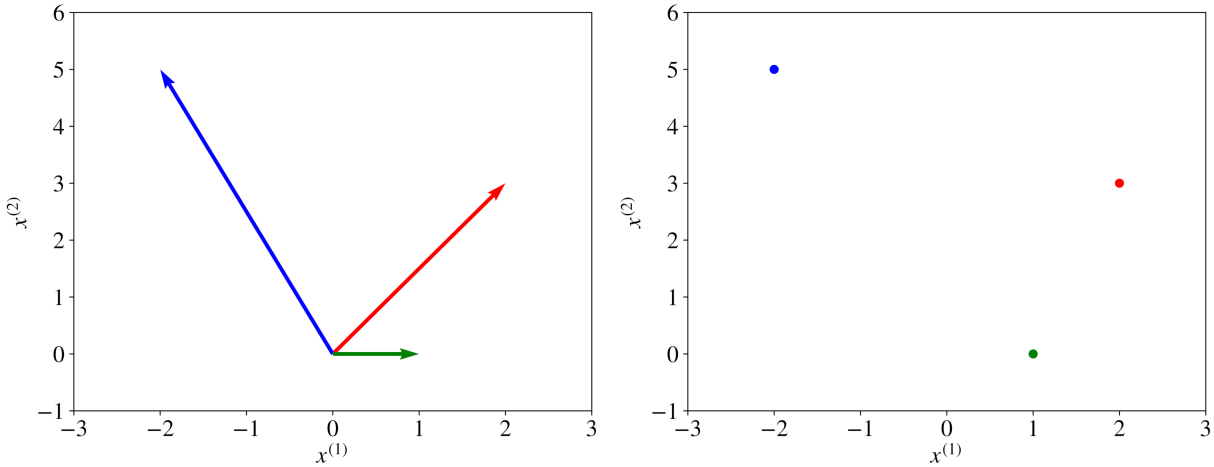


Figure 1: Three vectors visualized as directions and as points.

The notation $x^{(j)}$ should not be confused with the power operator, such as the 2 in x^2 (squared) or 3 in x^3 (cubed). If we want to apply a power operator, say squared, to an indexed attribute of a vector, we write like this: $(x^{(j)})^2$.

A variable can have two or more indices, like this: $x_i^{(j)}$ or like this $x_{i,j}^{(k)}$. For example, in neural networks, we denote as $x_{l,u}^{(j)}$ the input feature j of unit u in layer l.

A **matrix** is a rectangular array of numbers arranged in rows and columns. Below is an example of a matrix with two rows and three columns,

$$\mathbf{A} = \begin{bmatrix} 2 & -2 & 1 \\ 3 & 5 & 0 \end{bmatrix}.$$

Matrices are denoted with bold capital letters, such as $\mathbf{A}$ or $\mathbf{W}$. You can notice from the above example of matrix $\mathbf{A}$ that matrices can be seen as regular structures composed of vectors. Indeed, the columns of matrix $\mathbf{A}$ above are vectors $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ illustrated in Figure 1.

A **set** is an unordered collection of unique elements. We denote a set as a calligraphic capital character, for example, $\mathcal{S}$. A set of numbers can be finite (include a fixed amount of values). In this case, it is denoted using accolades, for example, $\{1, 3, 18, 23, 235\}$ or $\{x_1, x_2, x_3, x_4, \ldots, x_n\}$. Alternatively, a set can be infinite and include all values in some interval. If a set includes all values between a and b, including a and b, it is denoted using brackets as $[a, b]$. If the set doesn't include the values a and b, such a set is denoted using parentheses like this: (a, b). For example, the set $[0, 1]$ includes such values as 0, 0.0001, 0.25, 0.784, 0.9995, and 1.0. A special set denoted $\mathbb{R}$ includes all numbers from minus infinity to plus infinity.

When an element x belongs to a set $\mathcal{S}$, we write $x \in \mathcal{S}$. We can obtain a new set $\mathcal{S}_3$ as an **intersection** of two sets $\mathcal{S}_1$ and $\mathcal{S}_2$. In this case, we write $\mathcal{S}_3 \leftarrow \mathcal{S}_1 \cap \mathcal{S}_2$. For example $\{1, 3, 5, 8\} \cap \{1, 8, 4\}$ gives the new set $\{1, 8\}$.

We can obtain a new set $\mathcal{S}_3$ as a **union** of two sets $\mathcal{S}_1$ and $\mathcal{S}_2$. In this case, we write $\mathcal{S}_3 \leftarrow \mathcal{S}_1 \cup \mathcal{S}_2$. For example $\{1, 3, 5, 8\} \cup \{1, 8, 4\}$ gives the new set $\{1, 3, 5, 8, 4\}$.

The notation $|\mathcal{S}|$ means the size of set $\mathcal{S}$, that is, the number of elements it contains.

1.1.2 Capital Sigma Notation

The summation over a collection $\mathcal{X} = \{x_1, x_2, \ldots, x_{n-1}, x_n\}$ or over the attributes of a vector $\mathbf{x} = [x^{(1)}, x^{(2)}, \ldots, x^{(m-1)}, x^{(m)}]$ is denoted like this:

$$\sum_{i=1}^{n} x_i \stackrel{\text{def}}{=} x_1 + x_2 + \ldots + x_{n-1} + x_n, \text{ or else: } \sum_{j=1}^{m} x^{(j)} \stackrel{\text{def}}{=} x^{(1)} + x^{(2)} + \ldots + x^{(m-1)} + x^{(m)}.$$

The notation $\stackrel{\text{def}}{=}$ means "is defined as".

The **Euclidean norm** of a vector $\mathbf{x}$, denoted by $\|\mathbf{x}\|$, characterizes the "size" or the "length" of the vector. It's given by $\sqrt{\sum_{j=1}^{D} \left(x^{(j)}\right)^2}$.

The distance between two vectors $\mathbf{a}$ and $\mathbf{b}$ is given by the **Euclidean distance**:

$$\|a - b\| \stackrel{\text{def}}{=} \sqrt{\sum_{i=1}^{N} \left(a^{(i)} - b^{(i)}\right)^2}.$$

1.2 What is Machine Learning

Machine learning is a subfield of computer science that is concerned with building algorithms that, to be useful, rely on a collection of examples of some phenomenon. These examples can come from nature, be handcrafted by humans, or generated by another algorithm.

Machine learning can also be defined as the process of solving a practical problem by,

1) collecting a dataset, and
2) algorithmically training a **statistical model** based on that dataset.

That statistical model is assumed to be used somehow to solve the practical problem. To save keystrokes, I use the terms "learning" and "machine learning" interchangeably. For the same reason, I often say "model" referring to a statistical model.

Learning can be supervised, semi-supervised, unsupervised, and reinforcement.

1.2.1 Supervised Learning

In **supervised learning**, the data analyst works with a collection of **labeled examples** $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\}$. Each element $\mathbf{x}_i$ among N is called a **feature vector**. In computer science, a vector is a one-dimensional array. A one-dimensional array, in turn, is an ordered and indexed sequence of values. The length of that sequence of values, D, is called the vector's **dimensionality**.

A feature vector is a vector in which each dimension j from 1 to D contains a value that describes the example. Each such value is called a **feature** and is denoted as $x^{(j)}$. For instance, if each example $\mathbf{x}$ in our collection represents a person, then the first feature, $x^{(1)}$, could contain height in cm, the second feature, $x^{(2)}$, could contain weight in kg, $x^{(3)}$ could contain gender, and so on. For all examples in the dataset, the feature at position j in the feature vector always contains the same kind of information. It means that if $x_i^{(2)}$ contains weight in kg in some example $\mathbf{x}_i$, then $x_k^{(2)}$ will also contain weight in kg in every example $\mathbf{x}_k$, for all k from 1 to N. The **label** y_i can be either an element belonging to a finite set of **classes** $\{1, 2, \ldots, C\}$, or a real number, or a more complex structure, like a vector, a matrix, a tree, or a graph. Unless otherwise stated, in this book y_i is either one of a finite set of classes or a real number.[2] You can think of a class as a category to which an example belongs.

For instance, if your examples are email messages and your problem is spam detection, then you have two classes: spam and not_spam. In supervised learning, the problem of predicting a class is called **classification**, while the problem of predicting a real number is called **regression**. The value that has to be predicted by a supervised model is called a **target**. An example of regression is a problem of predicting the salary of an employee given their work experience and knowledge. An example of classification is when a doctor enters the characteristics of a patient into a software application, and the application returns the diagnosis.

The difference between classification and regression is shown in Figure 2. In classification, the learning algorithm looks for a line (or, more generally, a hypersurface) that separates examples of different classes from one another. In regression, on the other hand, the learning algorithm looks to find a line or a hypersurface that closely follows the training examples.

[2] A real number is a quantity that can represent a distance along a line. Examples: 0, −256.34, 1000, 1000.2.

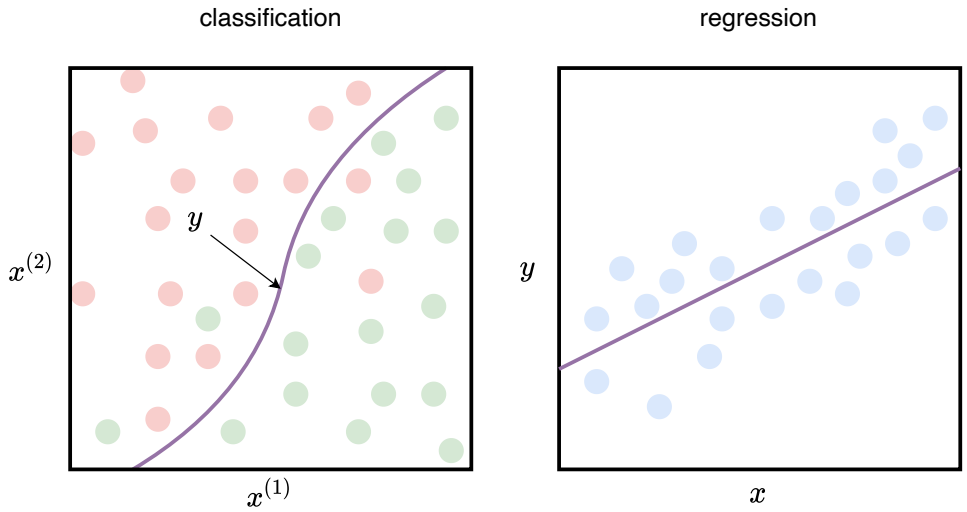


Figure 2: Difference between classification and regression.

The goal of a **supervised learning algorithm** is to use a dataset to produce a model that takes a feature vector $\mathbf{x}$ as input and outputs information that allows deducing a label for this feature vector. For instance, a model created using a dataset of patients could take as input a feature vector describing a patient and output a probability that the patient has cancer.

Even if the model is typically a mathematical function, when thinking about what the model does with the input, it is convenient to think that the model "looks" at the values of some features in the input and, based on experience with similar examples, outputs a value. That output value is a number or a class "the most similar" to the labels seen in the past in the examples with similar values of features. It looks simplistic, but the decision tree model and the k-nearest neighbors algorithm work almost like that.

1.2.2 Unsupervised Learning

In **unsupervised learning**, the dataset is a collection of **unlabeled examples** $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}$. Again, $\mathbf{x}$ is a feature vector, and the goal of an **unsupervised learning algorithm** is to create a model that takes a feature vector $\mathbf{x}$ as input and either transforms it into another vector or into a value that can be used to solve a practical problem. For example, in **clustering**, the model returns the ID of the cluster for each feature vector in the dataset. Clustering is useful for finding groups of similar objects in a large collection of objects, such as images or text documents. By using clustering, for example, the analyst can sample a sufficiently representative yet small subset of unlabeled examples from a large

collection of examples for manual labeling: a few examples are sampled from each cluster instead of sampling directly from the large collection and risking only sampling examples very similar to one another.

In **dimensionality reduction**, the model's output is a feature vector with fewer dimensions than the input. For example, the scientist has a feature vector that is too complex to visualize (it has more than three dimensions). The dimensionality reduction model can transform that feature vector into a new feature vector (by preserving the information up to some extent) with only two or three dimensions. This new feature vector can be plotted on a graph.

In **outlier detection**, the output is a real number that indicates how the input feature vector is different from a "typical" example in the dataset. Outlier detection is useful for solving a network intrusion problem (by detecting abnormal network packets that are different from a typical packet in "normal" traffic) or detecting novelty (such as a document different from the existing documents in a collection).

1.2.3 Semi-Supervised Learning

In **semi-supervised learning**, the dataset contains both labeled and unlabeled examples. Usually, the quantity of unlabeled examples is much higher than the number of labeled examples. The goal of a **semi-supervised learning algorithm** is the same as the goal of the supervised learning algorithm. The hope here is that, by using many unlabeled examples, a learning algorithm can find (we might say "produce" or "compute") a better model.

1.2.4 Reinforcement Learning

Reinforcement learning is a subfield of machine learning where the machine (called an agent) "lives' ' in an environment and is capable of perceiving the state of that environment as a vector of features. The machine can execute actions in non-terminal states. Different actions bring different rewards and could also move the machine to another state of the environment. A common goal of a reinforcement learning algorithm is to learn an optimal **policy**.

An optimal policy is a function (similar to the model in supervised learning) that takes the feature vector of a state as input and outputs an optimal action to execute in that state. The action is optimal if it maximizes the expected average long-term reward.

Reinforcement learning solves a particular problem where decision making is sequential, and the goal is long-term, such as game playing, robotics, resource management, or logistics.

In this book, for simplicity, most explanations are limited to supervised learning. However, all the material presented in the book is applicable to other types of machine learning.

1.3 Data and Machine Learning Terminology

Now let's introduce the common data terminology (such as data used directly and indirectly, raw and tidy data, training and holdout data) and the terminology related to machine learning (such as baseline, hyperparameter, pipeline, and others).

1.3.1 Data Used Directly and Indirectly

The data you will work with in your machine learning project can be used to form the examples **x directly** or **indirectly**.

Imagine that we build a named entity recognition system. The input of the model is a sequence of words; the output is the sequence of labels[3] of the same length as the input. To make the data readable by a machine learning algorithm, we have to transform each natural language word into a machine-readable array of attributes, which we call a feature vector.[4] Some features in the feature vector may contain the information that distinguishes that specific word from other words in the dictionary. Other features can contain additional attributes of the word in that specific sequence, such as its shape (lowercase, uppercase, capitalized, and so on). Or it can be binary attributes indicating whether this word is the first word of some human name or the last word of the name of some location or organization. To create these latter binary features, we may decide to use some dictionaries, lookup tables, gazetteers, or other machine learning models making predictions about words.

You could already have noticed that the collection of word sequences is the data used to form training examples directly, while the data contained in dictionaries, lookup tables, and gazetteers is used indirectly: we can use it to extend feature vectors with additional features, but we cannot use it to create new feature vectors.

1.3.2 Raw and Tidy Data

As we just discussed, directly used data is a collection of entities that constitute the basis of a dataset. Each entity in that collection can be transformed into a training example. **Raw data** is a collection of entities in their natural form; they cannot always be directly employable for machine learning. For instance, a Word document or a JPEG file are pieces of raw data; they cannot be directly used by a machine learning algorithm.[5]

[3] Labels can be, for example, values from the set {"Location", "Organization", "Person", "Other"}.

[4] The terms "attribute" and "feature" are often used interchangeably. In this book, I use the term "attribute" to describe a specific property of an example, while the term "feature" refers to value $x^{(j)}$ at position j in the feature vector $\mathbf{x}$ used by a machine learning algorithm.

[5] The term "unstructured data" is often used to designate a data element that contains information whose type was not formally defined. Examples of unstructured data are photos, images, videos, text messages, social media posts, PDFs, text documents, and emails. The term "semi-structured data" refers to data elements whose structure helps deriving types of some information encoded in those data elements. Examples of semi-structured data include log files, comma- and tab-delimited text files, as well as documents in JSON and XML formats.

To be employable in machine learning, a necessary (but not sufficient) condition for the data is to be tidy. **Tidy data** can be seen as a spreadsheet, in which each row represents one example, and columns represent various **attributes** of an example, as shown in Figure 3. Sometimes raw data can be tidy, e.g., provided to you in the form of a spreadsheet. However, in practice, to obtain tidy data from raw data, data analysts often resort to the procedure called **feature engineering**, which is applied to the direct and, optionally, indirect data with the goal to transform each raw example into a feature vector $\mathbf{x}$. Chapter 4 is devoted entirely to feature engineering.

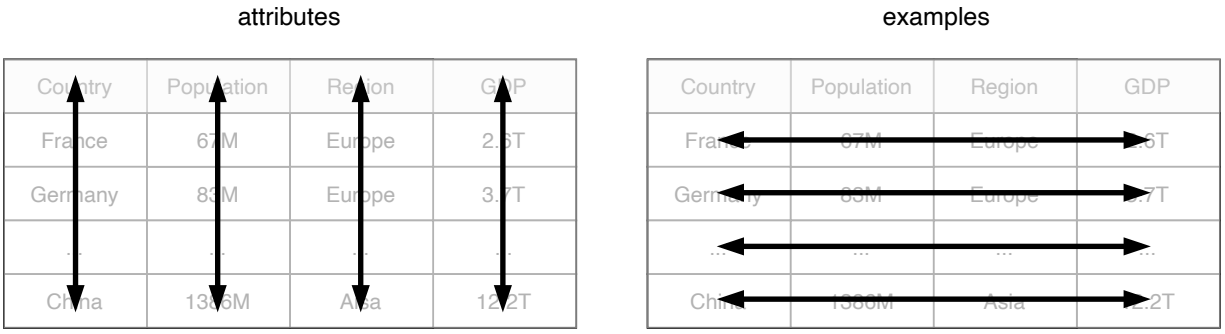


Figure 3: Tidy data: examples are rows and attributes are columns.

It's important to note here that for some tasks, an example used by a learning algorithm can have a form of a sequence of vectors, a matrix, or a sequence of matrices. The notion of data tidiness for such algorithms is defined similarly: you only replace "row of fixed width in a spreadsheet" by a matrix of fixed width and height, or a generalization of matrices to a higher dimension called a **tensor**.

The term "tidy data" was coined by Hadley Wickham in his paper with the same title.[6]

As I mentioned at the beginning of this subsection, data can be tidy, but still not usable by a particular machine learning algorithm. Most machine learning algorithms, in fact, only accept training data in the form of a collection of numerical feature vectors. Consider the data shown in Figure 3. The attribute "Region" is categorical and not numerical. The decision tree learning algorithm can work with categorical values of attributes, but most learning algorithms cannot. In Section **??** of Chapter 4, we will see how to transform a categorical attribute into a numerical feature.

Note that in the academic machine learning literature, the word "example" typically refers to a tidy data example with an optionally assigned label. However, during the stage of data collection and labeling, which we consider in the next chapter, examples can still be in the raw form: images, texts, or rows with categorical attributes in a spreadsheet. In this book, when it's important to highlight the difference, I will say **raw example** to indicate that a

[6]Wickham, Hadley. "Tidy data." Journal of Statistical Software 59.10 (2014): 1-23.

piece of data was not transformed into a feature vector yet. Otherwise, assume that examples have the form of feature vectors.

1.3.3 Training and Holdout Sets

In practice, data analysts work with three distinct sets of examples:

1) training set,
2) validation set,[7] and
3) test set.

Once you have got the data in the form of a collection of examples, the first thing you do in your machine learning project is shuffle the examples and split the dataset into three distinct sets: **training**, **validation**, and **test**. The training set is usually the biggest one; the learning algorithm uses the training set to produce the model. The validation and test sets are roughly the same size, much smaller than the size of the training set. The learning algorithm is not allowed to use examples from the validation or test sets to train the model. That is why those two sets are also called **holdout sets**.

The reason to have three sets, and not one, is simple: when we train a model, we don't want the model to only do well at predicting labels of examples the learning algorithm has already seen. A trivial algorithm that simply memorizes all training examples and then uses the memory to "predict" their labels will make no mistakes when asked to predict the labels of the training examples. However, such an algorithm would be useless in practice. What we really want is a model that is good at predicting examples that the learning algorithm didn't see. In other words, we want good performance on a holdout set.[8]

We need two holdout sets and not one because we use the validation set to 1) choose the learning algorithm, and 2) find the best configuration values for that learning algorithm (known as **hyperparameters**). We use the test set to assess the model before delivering it to the client or putting it in production. That is why it's important to make sure that no information from the validation or test sets is exposed to the learning algorithm. Otherwise, the validation and test results will most likely be too optimistic. This can indeed happen due to **data leakage**, an important phenomenon we consider in Section **??** of Chapter 3 and subsequent chapters.

[7] In some literature, the validation set can also be called "development set." Sometimes, when the labeled examples are scarce, analysts can decide to work without a validation set, as we will see in Chapter 5 in the section on **cross-validation**.

[8] To be precise, we want the model to do well on most random samples from the statistical distribution to which our data belongs. We assume that if the model demonstrates good performance on a holdout set, randomly drawn from the unknown distribution of our data, there are high chances that our model will do well on other random samples of our data.

1.3.4 Baseline

In machine learning, a **baseline** is a simple algorithm for solving a problem, usually based on a heuristic, simple summary statistics, randomization, or very basic machine learning algorithm. For example, if your problem is classification, you can pick a baseline classifier and measure its performance. This baseline performance will then become what you compare any future model to (usually, built using a more sophisticated approach).

1.3.5 Machine Learning Pipeline

A machine learning **pipeline** is a sequence of operations on the dataset that goes from its initial state to the model.

A pipeline can include, among others, such stages as data partitioning, missing data imputation, feature extraction, data augmentation, class imbalance reduction, dimensionality reduction, and model training.

In practice, when we deploy a model in production, we usually deploy an entire pipeline. Furthermore, an entire pipeline is usually optimized when hyperparameters are tuned.

1.3.6 Parameters vs. Hyperparameters

Hyperparameters are inputs of machine learning algorithms or pipelines that influence the performance of the model. They don't belong to the training data and cannot be learned from it. For example, the maximum depth of the tree in the decision tree learning algorithm, the misclassification penalty in support vector machines, k in the k-nearest neighbors algorithm, the target dimensionality in dimensionality reduction, and the choice of the missing data imputation technique are all examples of hyperparameters.

Parameters, on the other hand, are variables that define the model trained by the learning algorithm. Parameters are directly modified by the learning algorithm based on the training data. The goal of learning is to find such values of parameters that make the model optimal in a certain sense. Examples of parameters are w and b in the equation of linear regression $y = wx + b$. In this equation, x is the input of the model, and y is its output (the prediction).

1.3.7 Classification vs. Regression

Classification is a problem of automatically assigning a **label** to an **unlabeled example**. Spam detection is a famous example of classification.

In machine learning, the classification problem is solved by a **classification learning algorithm** that takes a collection of **labeled examples** as inputs and produces a **model** that can take an unlabeled example as input and either directly output a label or output a

number that can be used by the analyst to deduce the label. An example of such a number is a probability of an input data element to have a specific label.

In a classification problem, a label is a member of a finite set of **classes**. If the size of the set of classes is two ("sick"/"healthy", "spam"/"not_spam"), we talk about **binary classification** (also called **binomial** in some sources). **Multiclass classification** (also called **multinomial**) is a classification problem with three or more classes.[9]

While some learning algorithms naturally allow for more than two classes, others are by nature binary classification algorithms. There are strategies to turn a binary classification learning algorithm into a multiclass one. I talk about one of them, **one-versus-rest**, in Section **??** of Chapter 6.

Regression is a problem of predicting a real-valued quantity given an unlabeled example. Estimating house price valuation based on house features, such as area, number of bedrooms, location, and so on, is a famous example of regression.

The regression problem is solved by a **regression learning algorithm** that takes a collection of labeled examples as inputs and produces a model that can take an unlabeled example as input and output a target.

1.3.8 Model-Based vs. Instance-Based Learning

Most supervised learning algorithms are **model-based**. A typical **model** is a **support vector machine** (**SVM**). Model-based learning algorithms use the training data to create a model with **parameters** learned from the training data. In SVM, the two parameters are $\mathbf{w}$ (a vector) and b (a real number). After the model is trained, it can be saved on disk while the training data can be discarded.

Instance-based learning algorithms use the whole dataset as the model. One instance-based algorithm frequently used in practice is **k-Nearest Neighbors** (kNN). In classification, to predict a label for an input example, the kNN algorithm looks at the close neighborhood of the input example in the space of feature vectors and outputs the label that it saw most often in this close neighborhood.

1.3.9 Shallow vs. Deep Learning

A **shallow learning** algorithm learns the parameters of the model directly from the features of the training examples. Most machine learning algorithms are shallow. The notorious exceptions are **neural network** learning algorithms, specifically those that build neural networks with more than one **layer** between input and output. Such neural networks are called **deep neural networks**. In deep neural network learning (or, simply, **deep learning**),

[9] There's still one label per example, though.

contrary to shallow learning, most model parameters are learned not directly from the features of the training examples, but from the outputs of the preceding layers.

1.3.10 Training vs. Scoring

When we apply a machine learning algorithm to a dataset in order to obtain a model, we talk about **model training** or simply training.

When we apply a trained model to an input example (or, sometimes, a sequence of examples) in order to obtain a prediction (or, predictions) or to somehow transform an input, we talk about **scoring**.

1.4 When to Use Machine Learning

Machine learning is a powerful tool for solving practical problems. However, like any tool, it should be used in the right context. Trying to solve all problems using machine learning would be a mistake.

You should consider using machine learning in one of the following situations.

1.4.1 When the Problem Is Too Complex for Coding

In a situation where the problem is so complex or big that you cannot hope to write all the rules to solve it and where a partial solution is viable and interesting, you can try to solve the problem with machine learning.

One example is spam detection: it's impossible to write the code that will implement such a logic that will effectively detect spam messages and let genuine messages reach the inbox. There are just too many factors to consider. For instance, if you program your spam filter to reject all messages from people who are not in your contacts, you risk losing messages from someone who has got your business card at a conference. If you make an exception for messages containing specific keywords related to your work, you will probably miss a message from your child's teacher, and so on.

If you still decide to directly program a solution to that complex problem, with time, you will have in your programming code so many conditions and exceptions from those conditions that maintaining that code will eventually become infeasible. In this situation, training a classifier on examples "spam"/"not_spam" seems logical and the only viable choice.

Another difficulty for writing code to solve a problem lies in the fact that humans have a hard time with prediction problems based on input that has too many parameters; it's especially true when those parameters are **correlated** in unknown ways. For example, take the problem of predicting whether a borrower will repay a loan. Hundreds of numbers represent each borrower: age, salary, account balance, frequency of past payments, married or not, number

of children, make and year of the car, mortgage balance, and so on. Some of those numbers may be important to make the decision, some may be less important alone, but become more important if considered in combination with some other numbers.

Writing code that will make such decisions is hard because, even for an expert, it's not clear how to combine, in an optimal way, all the attributes describing a person into a prediction.

1.4.2 When the Problem Is Constantly Changing

Some problems may continuously change with time so that the programming code must be regularly updated. That results in the frustration of software engineers working on the problem, an increased chance of introducing errors, difficulties of combining "previous" and "new" logic, and significant overhead of testing and deploying updated solutions.

For example, you can have a task of scraping specific data elements from a collection of webpages. Let's say that for each webpage in that collection, you write a set of fixed data extraction rules in the following form: "pick the third <p> element from <body> and then pick the data from the second inside that <p>." If a website owner changes the design of a webpage, the data you scrape may end up in the second or the fourth <p> element, making your extraction rule wrong. If the collection of webpages you scrape is large (thousands of URLs), every day you will have rules that become wrong; you will end up endlessly fixing those rules. Needless to say that very few software engineers would love to do such work on a daily basis.

1.4.3 When It Is a Perceptive Problem

Today, it's hard to imagine someone trying to solve **perceptive problems** such as speech, image, and video recognition without using machine learning. Consider an image. It's represented by millions of pixels. Each pixel is given by three numbers: the intensity of red, green, and blue channels. In the past, engineers tried to solve the problem of image recognition (detecting what's on the picture) by applying handcrafted "filters" to square patches of pixels. If one filter, for example, the one that was designed to "detect" grass, generates a high value when applied to many pixel patches, while another filter, designed to detect brown fur, also returns high values for many patches, then we can say that there are high chances that the image represents a cow in a field (I'm simplifying a bit).

Today, perceptive problems are effectively solved using machine learning models, such as neural networks. We consider the problem of training neural networks in Chapter 6.

1.4.4 When It Is an Unstudied Phenomenon

If we need to be able to make predictions of some phenomenon that is not well-studied scientifically, but examples of it are observable, then machine learning might be an appropriate

(and, in some cases, the only available) option. For example, machine learning can be used to generate personalized mental health medication options based on the patient's genetic and sensory data. Doctors might not necessarily be able to interpret such data to make an optimal recommendation, while a machine can discover patterns in data by analyzing thousands of patients and predicting which molecule has the highest chance to help a given patient.

Another example of observable but unstudied phenomena are logs of a complex computing system or a network. Such logs are generated by multiple independent or interdependent processes. For a human, it's hard to make predictions about the future state of the system based on logs alone without having a model of each process and their interdependency. If the number of examples of historical log records is high enough (which is often the case), the machine can learn patterns hidden in logs and be able to make predictions without knowing anything about each process.

Finally, making predictions about people based on their observed behavior is hard. In this problem, we obviously cannot have a model of a person's brain, but we have readily available examples of expressions of the person's ideas (in the form of online posts, comments, and other activities). Based on those expressions alone, a machine learning model deployed in a social network can recommend the content or other people to connect with.

1.4.5 When the Problem Has a Simple Objective

Machine learning is especially suitable for solving problems that you can formulate as a problem with a simple objective: such as yes/no decisions or a single number. In contrast, you cannot use machine learning to build a model that works as a general video game, like Mario, or a word processing software, like Word. This is due to too many different decisions to make: what to display, where and when, what should happen as a reaction to the user's input, what to write to or read from the hard drive, and so on; getting examples that illustrate all (or even most) of those decisions is practically infeasible.

1.4.6 When It Is Cost-Effective

Three major sources of cost in machine learning are:

- collecting, preparing, and cleaning the data,
- training the model,
- building and running the infrastructure to serve and and monitor the model, as well as labor resources to maintain it.

The cost of training the model includes human labor and, in some cases, the expensive hardware needed to train deep models. Model maintenance includes continuously monitoring the model and collecting additional data to keep the model up to date.

1.5 When Not to Use Machine Learning

There are plenty of problems that cannot be solved using machine learning; it's hard to characterize all of them. Here we only consider several hints.

You probably should not use machine learning when:

- every action of the system or a decision made by it must be explainable,
- every change in the system's behavior compared to its past behavior in a similar situation must be explainable,
- the cost of an error made by the system is too high,
- you want to get to the market as fast as possible,
- getting the right data is too hard or impossible,
- you can solve the problem using traditional software development at a lower cost,
- a simple heuristic would work reasonably well,
- the phenomenon has too many outcomes while you cannot get a sufficient amount of examples to represent them (like in video games or word processing software),
- you build a system that will not have to be improved frequently over time,
- you can manually fill an exhaustive lookup table by providing the expected output for any input (that is, the number of possible input values is not too large, or getting outputs is fast and cheap).

1.6 What is Machine Learning Engineering

Machine learning engineering (MLE) is the use of scientific principles, tools, and techniques of machine learning and traditional software engineering to design and build complex computing systems. MLE encompasses all stages from data collection, to model training, to making the model available for use by the product or the customers.

Typically, a data analyst[10] is concerned with understanding the business problem, building a model to solve it, and evaluating the model in a restricted development environment. A machine learning engineer, in turn, is concerned with sourcing the data from various systems and locations and preprocessing it, programming features, training an effective model that will run in the production environment, coexist well with other production processes, be stable, maintainable, and easily accessible by different types of users with different use cases.

In other words, MLE includes any activity that lets machine learning algorithms be implemented as a part of an effective production system.

In practice, machine learning engineers might be employed in such activities as rewriting a

[10]Since circa 2013, data scientist has become a popular job title. Unfortunately, companies and experts don't have an agreement on the definition of the term. Instead, I use the term "data analyst" by referring to a person capable of applying numerical or statistical analysis to data ready for analysis.

data analyst's code from rather slow R and Python[11] into more efficient Java or C++, scaling this code and making it more robust, packaging the code into an easy-to-deploy versioned package, optimizing the machine learning algorithm to make sure that it generates a model compatible with, and running correctly in, the organization's production environment.

In many organizations, data analysts execute some of the MLE tasks, such as data collection, transformation, and feature engineering. On the other hand, machine learning engineers often execute some of the data analysis tasks, including learning algorithm selection, hyperparameter tuning, and model evaluation.

Working on a machine learning project is different from working on a typical software engineering project. Unlike traditional software, where a program's behavior usually is deterministic, machine learning applications incorporate models whose behavior may naturally degrade over time, or they can start behaving abnormally. Such abnormal behavior of the model might be explained by various reasons, including a fundamental change in the input data or an updated feature extractor that now returns a different distribution of values or values of a different type. They often say that machine learning systems "fail silently." A machine learning engineer must be capable of preventing such failures or, when it's impossible to prevent them entirely, know how to detect and handle them when they happen.

1.7 Machine Learning Project Life Cycle

A machine learning project starts with understanding the business objective. Usually, a business analyst works with the client[12] and the data analyst to transform a business problem into an engineering project. The engineering project may or may not have a machine learning part. In this book, we, of course, consider engineering projects that have some machine learning involved.

Once an engineering project is defined, this is where the scope of the machine learning engineering starts. In the scope of a broader engineering project, machine learning must first have a well-defined **goal**. The goal of machine learning is a specification of what a statistical model receives as input, what it generates as output, and the criteria of acceptable (or unacceptable) behavior of the model.

The goal of machine learning is not necessarily the same as the business objective. The business objective is what the organization wants to achieve. For example, the business objective of Google with Gmail can be to make Gmail the most-used email service in the world. Google might create multiple machine learning engineering projects to achieve that business objective. The goal of one of those machine learning projects can be to distinguish Primary emails from Promotions with accuracy above 90%.

[11] Many scientific modules in Python are indeed implemented in fast C/C++; however, data analyst's own Python code can still be slow.

[12] If the machine learning project supports a product developed and sold by the organization, then the business analyst works with the product owner.

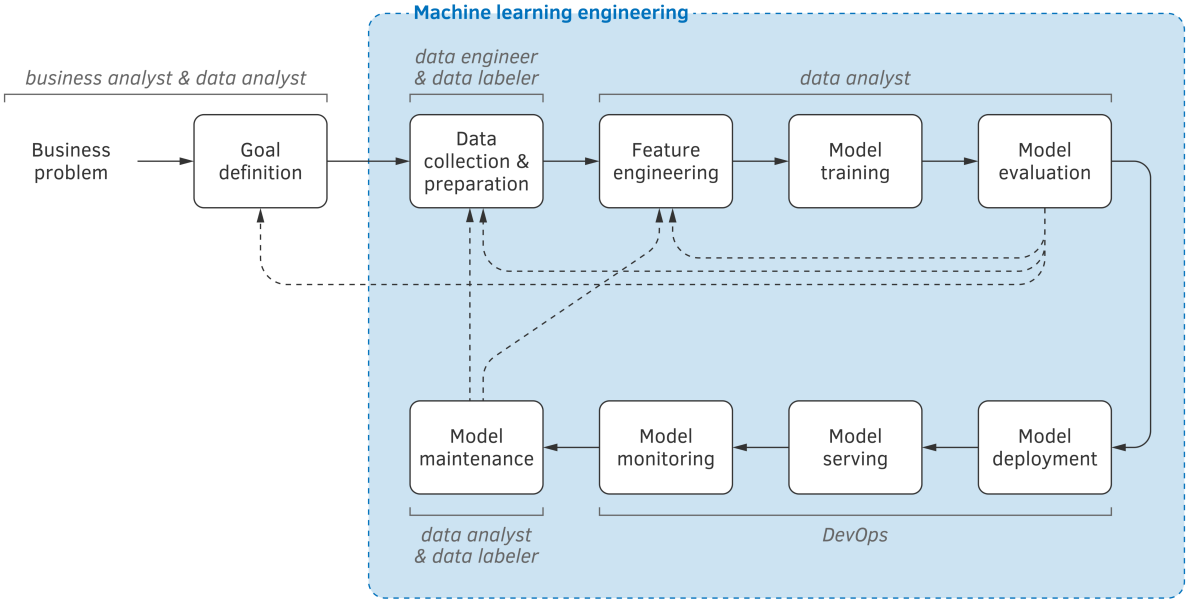


Figure 4: Machine learning project life cycle.

Overall, a machine learning project life cycle, illustrated in Figure 4, consists of the following stages: 1) goal definition, 2) data collection and preparation, 3) feature engineering, 4) model training, 5) model evaluation, 6) model deployment, 7) model serving, 8) model monitoring, and 9) model maintenance.

In Figure 4, the scope of machine learning engineering (and the scope of this book) is limited by the blue zone. The solid arrows show a typical flow of the project stages. The dashed arrows indicate that at some stages, a decision can be made to go back in the process and either collect more data or collect different data, and revise features (by decommissioning some of them and engineering new ones).

Every stage mentioned above will be considered in one of the book's chapters. But first, let's discuss how to prioritize machine learning projects, define the project's goal, and structure a machine learning team. The next chapter is devoted to these three questions.

1.8 Summary

A model-based machine learning algorithm takes a collection of training examples as input and outputs a model. An instance-based machine learning algorithm uses the entire training

dataset as a model. The training data is exposed to the machine learning algorithm, while holdout data isn't.

A supervised learning algorithm builds a model that takes a feature vector and outputs a prediction about that feature vector. An unsupervised learning algorithm builds a model that takes a feature vector as input and transforms it into something useful.

Classification is the problem of predicting, for an input example, one of a finite set of classes. Regression, in turn, is a problem of predicting a numerical target.

Data can be used directly or indirectly. Directly-used data is a basis for forming a dataset of examples. Indirectly-used data is used to enrich those examples.

The data for machine learning must be tidy. A tidy dataset can be seen as a spreadsheet where each row is an example, and each column is one of the properties of an example. In addition to being tidy, most machine learning algorithms require numerical data, as opposed to categorical. Feature engineering is the process of transforming data into a form that machine learning algorithms can use.

A baseline is essential to make sure that the model works better than a simple heuristic.

In practice, machine learning is implemented as a pipeline that contains chained stages of data transformation, from data partitioning to missing-data imputation, to class imbalance and dimensionality reduction, to model training. The hyperparameters of the entire pipeline are usually optimized; the entire pipeline can be deployed and used for predictions.

Parameters of the model are optimized by the learning algorithm based on the training data. The values of hyperparameters cannot be learned by the learning algorithm and are, in turn, tuned by using the validation dataset. The test set is only used to assess the model's performance and report it to the client or product owner.

A shallow learning algorithm trains a model that makes predictions directly from the input features. A deep learning algorithm trains a layered model, in which each layer generates outputs by taking the outputs of the preceding layer as inputs.

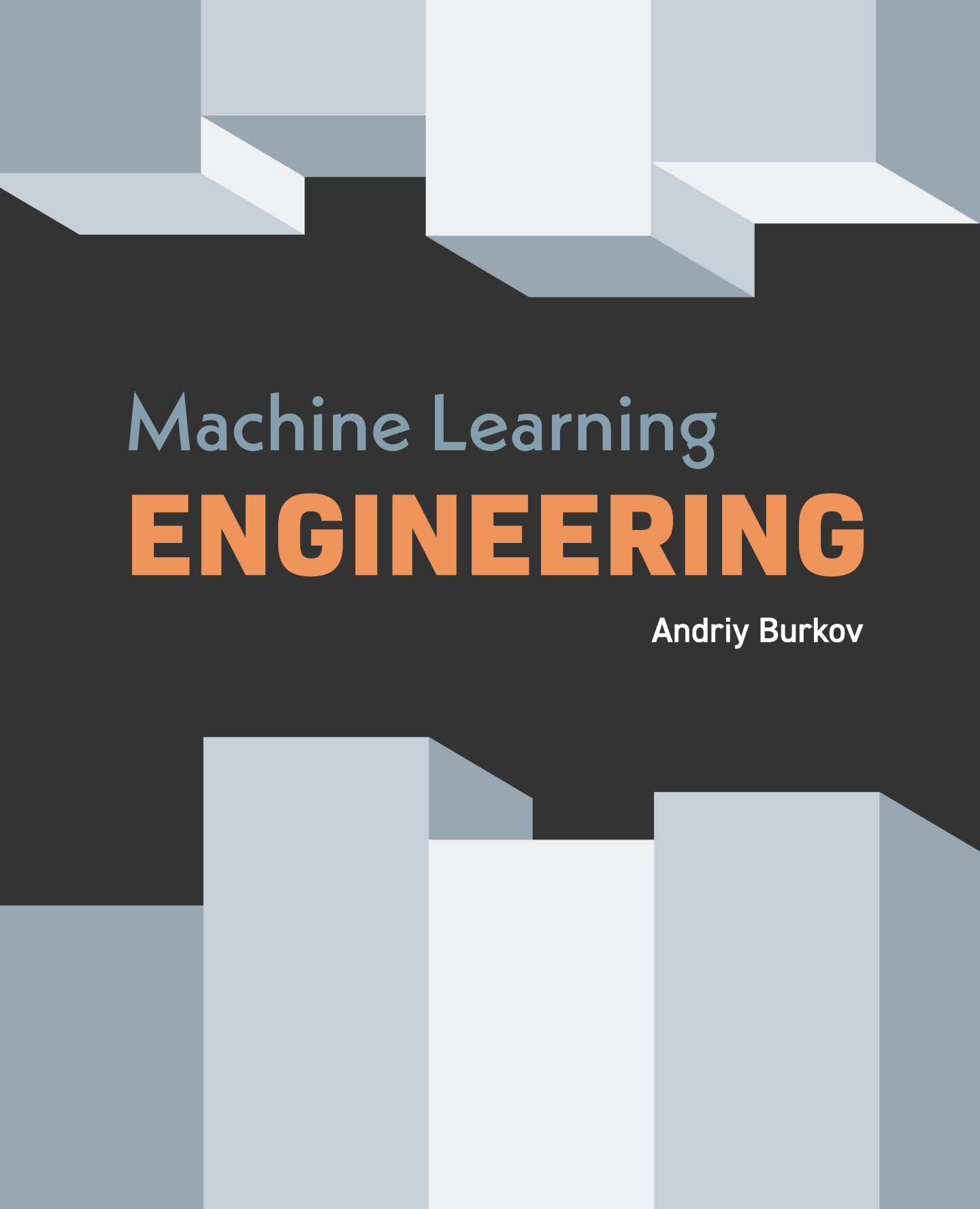
You should consider using machine learning to solve a business problem when the problem is too complex for coding, the problem is constantly changing, it is a perceptive problem, it is an unstudied phenomenon, the problem has a simple objective, and it is cost-effective.

There are many situations when machine learning should, probably, not be used: when explainability is needed, when errors are intolerable, when traditional software engineering is a less expensive option, when all inputs and outputs can be enumerated and saved in a database, and when data is hard to get or too expensive.

Machine learning engineering (MLE) is the use of scientific principles, tools, and techniques of machine learning and traditional software engineering to design and build complex computing systems. MLE encompasses all stages from data collection, to model training, to making the model available for use by the product or the consumers.

A machine learning project life cycle consists of the following stages: 1) goal definition, 2) data collection and preparation, 3) feature engineering, 4) model training, 5) model evaluation, 6) model deployment, 7) model serving, 8) model monitoring, and 9) model maintenance.

Every stage will be considered in one of the book's chapters.


Machine Learning
ENGINEERING
Andriy Burkov

“In theory, there is no difference between theory and practice. But in practice, there is.”

— Benjamin Brewster

“The perfect project plan is possible if one first documents a list of all the unknowns.”

— Bill Langley

“When you’re fundraising, it’s AI. When you’re hiring, it’s ML. When you’re implementing, it’s linear regression. When you’re debugging, it’s printf().”

— Baron Schwartz

The book is distributed on the “read first, buy later” principle.

2 Before the Project Starts

Before a machine learning project starts, it must be prioritized. Prioritization is inevitable: the team and equipment capacity is limited, while the organization's backlog of projects could be very long.

To prioritize a project, one has to estimate its complexity. With machine learning, accurate complexity estimation is rarely possible because of major unknowns, such as whether the required model quality is attainable in practice, how much data is needed, and what, and how many features are necessary.

Furthermore, a machine learning project must have a well-defined goal. Based on the goal of the project, the team could be adequately adjusted and resources provisioned.

In this chapter, we consider these and related activities that must be taken care of before a machine learning project starts.

2.1 Prioritization of Machine Learning Projects

The key considerations in the prioritization of a machine learning project, are impact and cost.

2.1.1 Impact of Machine Learning

The impact of using machine learning in a broader engineering project is high when, 1) machine learning can replace a complex part in your engineering project or 2) there's a great benefit in getting inexpensive (but probably imperfect) predictions.

For example, a complex part of an existing system can be rule-based, with many nested rules and exceptions. Building and maintaining such a system can be extremely difficult, time-consuming, and error-prone. It can also be a source of significant frustration for software engineers when they are asked to maintain that part of the system. Can the rules be learned instead of programming them? Can an existing system be used to generate labeled data easily? If yes, such a machine learning project would have a high impact and low cost.

Inexpensive and imperfect predictions can be valuable, for example, in a system that dispatches a large number of requests. Let's say many such requests are "easy" and can be solved quickly using some existing automation. The remaining requests are considered "difficult" and must be addressed manually.

A machine learning-based system that recognizes "easy" tasks and dispatches them to the automation will save a lot of time for humans who will only concentrate their effort and time on difficult requests. Even if the dispatcher makes an error in prediction, the difficult request will reach the automation, the automation will fail on it, and the human will eventually receive that request. If the human gets an easy request by mistake, there's no problem either: that easy request can still be sent to the automation or processed by the human.

2.1.2 Cost of Machine Learning

Three factors highly influence the cost of a machine learning project:

- the difficulty of the problem,
- the cost of data, and
- the need for accuracy.

Getting the right data in the right amount can be very costly, especially if manual labeling is involved. The need for high accuracy can translate into the requirement of getting more data or training a more complex model, such as a unique **architecture** of a deep neural network or a nontrivial **ensembling** architecture.

When you think about the problem's difficulty, the primary considerations are:

- whether an implemented algorithm or a software library capable of solving the problem is available (if yes, the problem is greatly simplified),
- whether significant computation power is needed to build the model or to run it in the production environment.

The second driver of the cost is data. The following considerations have to be made:

- can data be generated automatically (if yes, the problem is greatly simplified),
- what is the cost of manual **annotation** of the data (i.e., assigning labels to unlabeled examples),
- how many examples are needed (usually, that cannot be known in advance, but can be estimated from known published results or the organization's own experience).

Finally, one of the most influential cost factors is the desired accuracy of the model. The machine learning project's cost grows superlinearly with the accuracy requirement, as illustrated in Figure 1. Low accuracy can also be a source of significant loss when the model is deployed in the production environment. The considerations to make:

- how costly is each wrong prediction, and
- what is the lowest accuracy level below which the model becomes impractical.

2.2 Estimating Complexity of a Machine Learning Project

There is no standard complexity estimation method for a machine learning project, other than by comparison with other projects executed by the organization or reported in the literature.

2.2.1 The Unknowns

There are several major unknowns that are almost impossible to guess with confidence unless you worked on a similar project in the past or read about such a project. The unknowns are:

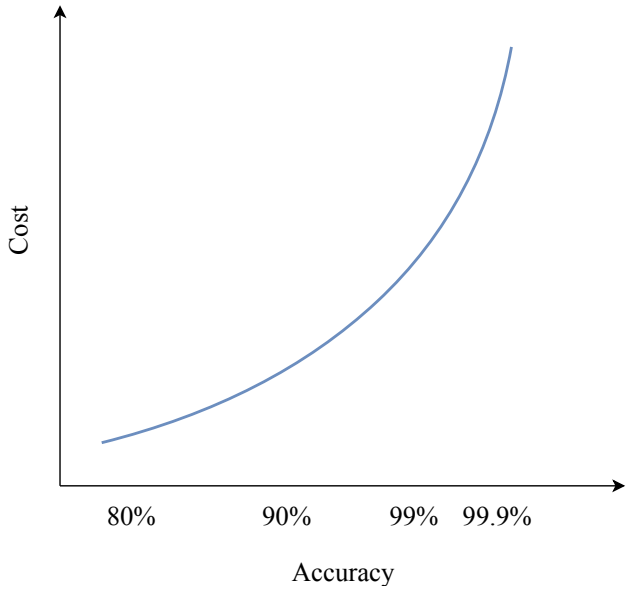


Figure 1: Superlinear growth of the cost as a function of accuracy requirement.

- whether the required quality is attainable in practice,
- how much data you will need to reach the required quality,
- what features and how many features are necessary so that the model can learn and generalize sufficiently,
- how large the model should be (especially relevant for neural networks and ensemble architectures), and
- how long will it take to train one model (in other words, how much time is needed to run one **experiment**) and how many experiments will be required to reach the desired level of performance.

One thing you can almost be sure of: if the required level of model **accuracy** (one of the popular model quality metrics we consider in Section **??** of Chapter 5) is above 99%, you can expect complications related to an insufficient quantity of labeled data. In some problems, even 95% accuracy is considered very hard to reach. (Here we assume, of course, that the data is balanced, that is, there's no **class imbalance**. We will discuss class imbalance in Section **??** of the next chapter.)

Another useful reference is the human performance on the task. This is typically a hard problem if you want your model to perform as well as a human.

2.2.2 Simplifying the Problem

One way to make a more educated guess is to simplify the problem and solve a simpler problem first. For example, assume that the problem is that of classifying a set of documents into 1000 topics. Run a pilot project by focusing on 10 topics first, by considering documents belonging to other 990 topics as "Other."[1] Manually label the data for these 11 classes (10 real topics, plus "Other"). The logic here is that it's much simpler for a human to keep in mind the definitions of only 10 topics compared to memorizing the difference between 1000 topics[2].

Once you have simplified your problem to 11 classes, solve it, and measure time on every stage. Once you see that the problem for 11 classes is solvable, you can reasonably hope that it will be solvable for 1000 classes as well. Your saved measurements can then be used to estimate the time required to solve the full problem, though you cannot simply multiply this time by 100 to get an accurate estimate. The quantity of data needed to learn to distinguish between more classes usually grows superlinearly with the number of classes.

An alternative way of obtaining a simpler problem from a potentially complex one is to split the problem into several simple ones by using the natural slices in the available data. For example, let an organization have customers in multiple locations. If we want to train a model that predicts something about the customers, we can try to solve that problem only for one location, or for customers in a specific age range.

2.2.3 Nonlinear Progress

The progress of a machine learning project is nonlinear. The prediction error usually decreases fast in the beginning, but then the progress gradually slows down.[3] Sometimes you see no progress and decide to add additional features that could potentially depend on external databases or knowledge bases. While you are working on a new feature or labeling more data (or outsourcing this task), no progress in model performance is happening.

Because of this nonlinearity of progress, you should make sure that the product owner (or the client) understands the constraints and risks. Carefully log every activity and track the time it took. This will help not only in reporting, but also in the estimation of the complexity of similar projects in the future.

[1] Putting examples belonging to 990 classes in one class will likely create a highly imbalanced dataset. If it's the case, you would prefer to **undersample** the data in the class "Other." We consider data undersampling in Section **??** of the next chapter.

[2] To save even more time, apply clustering to the whole collection of unlabeled documents and only manually label documents belonging to one or a few clusters.

[3] The 80/20 rule of thumb often applies: 80% of progress is made using the first 20% of resources.

2.3 Defining the Goal of a Machine Learning Project

The **goal** of a machine learning project is to build a model that solves, or helps solve, a business problem. Within a project, the model is often seen as a black box described by the structure of its input (or inputs) and output (or outputs), and the minimum acceptable level of performance (as measured by accuracy of prediction or another **performance metric**).

2.3.1 What a Model Can Do

The model is typically used as a part of a system that serves some purpose. In particular, the model can be used within a broader system to:

- automate (for example, by taking action on the user's behalf or by starting or stopping a specific activity on a server),
- alert or prompt (for example, by asking the user if an action should be taken or by asking a system administrator if the traffic seems suspicious),
- organize, by presenting a set of items in an order that might be useful for a user (for example, by sorting pictures or documents in the order of similarity to a query or according to the user's preferences),
- annotate (for instance, by adding contextual annotations to displayed information, or by highlighting, in a text, phrases relevant to the user's task),
- extract (for example, by detecting smaller pieces of relevant information in a larger input, such as named entities in the text: proper names, companies, or locations),
- recommend (for example, by detecting and showing to a user highly relevant items in a large collection based on item's content or user's reaction to the past recommendations),
- classify (for example, by dispatching input examples into one, or several, of a predefined set of distinctly-named groups),
- quantify (for example, by assigning a number, such as a price, to an object, such as a house),
- synthesize (for example, by generating new text, image, sound, or another object similar to the objects in a collection),
- answer an explicit question (for example, "Does this text describe that image?" or "Are these two images similar?"),
- transform its input (for example, by reducing its dimensionality for visualization purposes, paraphrasing a long text as a short abstract, translating a sentence into another language, or augmenting an image by applying a filter to it),
- detect a novelty or an anomaly.

Almost any business problem solvable with machine learning can be defined in a form similar to one from the above list. If you cannot define your business problem in such a form, likely, machine learning is not the best solution in your case.

2.3.2 Properties of a Successful Model

A successful model has the following four properties:

- it respects the input and output specifications and the performance requirement,
- it benefits the organization (measured via cost reduction, increased sales or profit),
- it helps the user (measured via productivity, engagement, and sentiment),
- it is scientifically rigorous.

A scientifically rigorous model is characterized by a predictable behavior (for the input examples that are similar to the examples that were used for training) and is reproducible. The former property (predictability) means that if input feature vectors come from the same distribution of values as the training data, then the model, on average, has to make the same percentage of errors as observed on the holdout data when the model was trained. The latter property (reproducibility) means that a model with similar properties can be easily built once again from the same training data using the same algorithm and values of hyperparameters. The word "easily" means that no additional analysis, labeling, or coding is necessary to rebuild the model, only the compute power.

When defining the goal of machine learning, make sure you solve the right problem. To give an example of an incorrectly defined goal, imagine your client has a cat and a dog and needs a system that lets their cat in the house but keeps their dog out. You might decide to train the model to distinguish cats from dogs. However, this model will also let *any cat* in and not just *their* cat. Alternatively, you may decide that because the client only has two animals, you will train a model that distinguishes between those two. In this case, because your classification model is binary, a raccoon will be classified as either the dog or the cat. If it's classified as the cat, it will be let in the house.[4]

Defining a single goal for a machine learning project could be challenging. Usually, within an organization, there will be multiple stakeholders having interest towards your project. An obvious stakeholder is the product owner. Let their objective be to increase the time the user spends on an online platform by at least 15%. At the same time, the executive VP would like to increase the revenue from advertisements by 20%. Furthermore, the finance team would like to reduce the monthly cloud bill by 10%. When defining the goal of your machine learning project, you should find the right balance between those possibly conflicting requirements and translate them into the choice of the model's input and output, **cost function**, and **performance metric**.

2.4 Structuring a Machine Learning Team

There are two cultures of structuring a machine learning team, depending on the organization.

[4]This is why having the class "Other" your classification problems is almost always a good idea.

2.4.1 Two Cultures

One culture says that a machine learning team has to be composed of data analysts who collaborate closely with software engineers. In such a culture, a software engineer doesn't need to have deep expertise in machine learning, but has to understand the vocabulary of their fellow data analysts.

According to other culture, all engineers in a machine learning team must have a combination of machine learning and software engineering skills.

There are pros and cons in each culture. The proponents of the former say that each team member must be the best in what they do. A data analyst must be an expert in many machine learning techniques and have a deep understanding of the theory to come up with an effective solution to most problems, fast and with minimal effort. Similarly, a software engineer must have a deep understanding of various computing frameworks and be capable of writing efficient and maintainable code.

The proponents of the latter say that scientists are hard to integrate with software engineering teams. Scientists care more about how accurate their solution is and often come up with solutions that are impractical and cannot be effectively executed in the production environment. Also, because scientists don't usually write efficient, well-structured code, the latter has to be rewritten into production code by a software engineer; depending on the project, that can turn out to be a daunting task.

2.4.2 Members of a Machine Learning Team

Besides machine learning and software engineering skills, a machine learning team may include experts in data engineering (also known as data engineers) and experts in data labeling.

Data engineers are software engineers responsible for ETL (for Extract, Transform, Load). These three conceptual steps are part of a typical data pipeline. Data engineers use ETL techniques and create an automated pipeline, in which raw data is transformed into analysis-ready data. Data engineers design how to structure the data and how to integrate it from various resources. They write on-demand queries on that data, or wrap the most frequent queries into fast application programming interfaces (APIs) to make sure that the data is easily accessible by analysts and other data consumers. Typically, data engineers are not expected to know any machine learning.

In most big companies, data engineers work separately from machine learning engineers in a data engineering team.

Experts in data labeling are responsible for four activities:

- manually or semi-automatically assign labels to unlabeled examples according to the specification provided by data analysts,
- build labeling tools,

- manage outsourced labelers, and
- validate labeled examples for quality.

A **labeler** is person responsible for assigning labels to unlabeled examples. Again, in big companies, data labeling experts may be organized in two or three different teams: one or two teams of labelers (for example, one local and one outsourced) and a team of software engineers, plus a user experience (UX) specialist, responsible for building labeling tools.

When possible, invite domain experts to work closely with scientists and engineers. Employ domain experts in your decision making about the inputs, outputs, and features of your model. Ask them what they think your model should predict. Just the fact that the data you can get access to can allow you to predict some quantity doesn't mean the model will be useful for the business.

Discuss with the domain experts what they look for in the data to make a specific business decision; that will help you with feature engineering. Discuss also what clients pay for and what is a deal-breaker for them; that will help you to translate a business problem into a machine learning problem.

Finally, there are DevOps engineers. They work closely with machine learning engineers to automate model deployment, loading, monitoring, and occasional or regular model maintenance. In smaller companies and startups, a DevOps engineer may be part of the machine learning team, or a machine learning engineer could be responsible for the DevOps activities. In big companies, DevOps engineers employed in machine learning projects usually work in a larger DevOps team. Some companies introduced the MLOps role, whose responsibility is to deploy machine learning models in production, upgrade those models, and build data processing pipelines involving machine learning models.

2.5 Why Machine Learning Projects Fail

According to various estimates made between 2017 and 2020, from 74% to 87% of machine learning and advanced analytics projects fail or don't reach production. The reasons for a failure range from organizational to engineering. In this section, we consider the most impactful of them.

2.5.1 Lack of Experienced Talent

As of 2020, both data science and machine learning engineering are relatively new disciplines. There's still no standard way to teach them. Most organizations don't know how to hire experts in machine learning and how to compare them. Most of the available talent on the market are people who completed one or several online courses and who don't possess significant practical experience. A significant fraction of the workforce has superficial expertise in machine learning obtained on toy datasets in a classroom context. Many don't have experience with the entire machine learning project life cycle. On the other hand, experienced software engineers that

might exist in an organization don't have expertise in handling data and machine learning models appropriately.

2.5.2 Lack of Support by the Leadership

As discussed in the previous section on the two cultures, scientists and software engineers often have different goals, motivations, and success criteria. They also work very differently. In a typical Agile organization, software engineering teams work in sprints with clearly defined expected deliverables and little uncertainty.

Scientists, on the other hand, work in high uncertainty and move ahead with multiple experiments. Most of such experiments don't result in any deliverable and, thus, can be seen by inexperienced leaders as no progress. Sometimes, after the model is built and deployed, the entire process has to start over because the model doesn't result in the expected increase of the metric the business cares about. Again, this can lead to the perception of the scientist's work by the leadership as wasted time and resources.

Furthermore, in many organizations, leaders responsible for data science and artificial intelligence (AI), especially at the vice-president level, have a non-scientific or even non-engineering background. They don't know how AI works, or have a very superficial or overly optimistic understanding of it drawn from popular sources. They might have such a mindset that with enough resources, technical and human, AI can solve any problem in a short amount of time. When fast progress doesn't happen, they easily blame scientists or entirely lose interest in AI as an ineffective tool with hard-to-predict and uncertain results.

Often, the problem lies in the inability of scientists to communicate the results and challenges to upper management. Because they don't share the vocabulary and have very different levels of technical expertise, even a success presented badly can be seen as a failure.

This is why, in successful organizations, data scientists are good popularizers, while top-level managers, responsible for AI and analytics, often have a technical or scientific background.

2.5.3 Missing Data Infrastructure

Data analysts and scientists work with data. The quality of the data is crucial for a machine learning project's success. Enterprise data infrastructure must provide the analyst with simple ways to get quality data for training models. At the same time, the infrastructure must make sure that similar quality data will be available once the model is deployed in production.

However, in practice, this is often not the case. Scientists obtain the data for training by using various ad-hoc scripts; they also use different scripts and tools to combine various data sources. Once the model is ready, it turns out that it's impossible, by using the available production infrastructure, to generate input examples for the model fast enough (or at all). We extensively talk about storing data and features in Chapters 3 and 4.

2.5.4 Data Labeling Challenge

In most machine learning projects, analysts use labeled data. This data is usually custom, so labeling is executed specifically for each project. As of 2019, according to some reports,[5] as many as 76% AI and data science teams label training data on their own, while 63% build their own labeling and annotation automation technology.

This results in a significant time spent by skilled data scientists on data labeling and labeling tool development. This is a major challenge for the effective execution of an AI project.

Some companies outsource data labeling to third-party vendors. However, without proper quality validation, such labeled data can turn out to be of low quality or entirely wrong. Organizations, in order to maintain quality and consistency across datasets, have to invest in formal and standardized training of internal or third-party labelers. This, in turn, can slow down machine learning projects. Though, according to the same reports, companies that outsource data labeling are more likely to get their machine learning projects up to production.

2.5.5 Siloed Organizations and Lack of Collaboration

Data needed for a machine learning project often resides within an organization in different places with different ownership, security constraints, and in different formats. In siloed organizations, people responsible for different data assets might not know one another. Lack of trust and collaboration results in friction when one department needs access to the data stored in a different department. Furthermore, different branches of one organization often have their own budgets, so collaboration becomes complicated because no side has an interest in spending their budget helping to the other side.

Even within one branch of an organization, there are often several teams involved in a machine learning project at different stages. For example, the data engineering team provides access to the data or individual features, the data science team works on modeling, ETL or DevOps work on the engineering aspects of deployment and monitoring, while the automation and internal tools teams develop tools and processes for a continuous model update. Lack of collaboration between any pair of the involved teams might result in the project being frozen for a long time. Typical reasons for mistrust between teams is the lack of understanding by the engineers of the tools and approaches used by the scientists and the lack of knowledge (or plain ignorance) by the scientists of software engineering good practices and design patterns.

2.5.6 Technically Infeasible Projects

Because of the high cost of many machine learning projects (due to high expertise and infrastructure cost) some organizations, to "recoup the investment," might target very ambitious goals: to completely transform the organization or the product or provide unrealistic

[5] Alegion and Dimensional Research, "What data scientists tell us about AI model training today," 2019.

return or investment. This results in very large-scale projects, involving collaboration between multiple teams, departments, and third-parties, and pushing those teams to their limits.

As a result, such overly ambitious projects could take months or even years to complete; some key players, including leaders and key scientists, might lose interest in the project or even leave the organization. The project could eventually be deprioritized, or, even when completed, be too late to the market. It is best, at least in the beginning, to focus on achievable projects, involving simple collaboration between teams, easy to scope, and targeting a simple business objective.

2.5.7 Lack of Alignment Between Technical and Business Teams

Many machine learning projects start without a clear understanding, by the technical team, of the business objective. Scientists usually frame the problem as classification or regression with a technical objective, such as high accuracy or low mean squared error. Without continuous feedback from the business team on the achievement of a business objective (such as an increased click-through rate or user retention), the scientists often reach an initial level of model performance (according to the technical objective), and then they are not sure if they are making any useful progress and whether an additional effort is worth it. In such situations, the projects end up being put on the shelf because time and resources were spent but the business team didn't accept the result.

2.6 Summary

Before a machine learning project starts, it must be prioritized, and the team working on the project must be built. The key considerations in the prioritization of a machine learning project, are impact and cost.

The impact of using machine learning is high when, 1) machine learning can replace a complex part in your engineering project, or 2) there's a great benefit in getting inexpensive (but probably imperfect) predictions.

The cost of the machine learning project is highly influenced by three factors: 1) the difficulty of the problem, 2) the cost of data, and 3) the needed model performance quality.

There is no standard method of estimation of how complex a machine learning project is other than by comparison with other projects executed by the organization or reported in the literature. There are several major unknowns that are almost impossible to guess: whether the required level of model performance is attainable in practice, how much data you will need to reach that level of performance, what features and how many features are needed, how large the model should be, and how long will it take to run one experiment and how many experiments will be needed to reach the desired level of performance.

One way to make a more educated guess is to simplify the problem and solve a simpler one.

The progress of a machine learning project is nonlinear. The error usually decreases fast in the beginning, but then the progress slows down. Because of this nonlinearity of progress, it's better to make sure that the client understands the constraints and the risks. Carefully log every activity and track the time it took. This will help not only in reporting but also in estimating complexity for similar projects in the future.

The goal of a machine learning project is to build a model that solves some business problem. In particular, the model can be used within a broader system to automate, alert or prompt, organize, annotate, extract, recommend, classify, quantify, synthesize, answer an explicit question, transform its input, and detect novelty or an anomaly. If you cannot frame the goal of machine learning in one of these forms, likely, machine learning is not the best solution.

A successful model 1) respects the input and output specifications and the minimum performance requirement, 2) benefits the organization and the user, and 3) is scientifically rigorous.

There are two cultures of structuring a machine learning team, depending on the organization. One culture says that a machine learning team has to be composed of data analysts who collaborate closely with software engineers. In such a culture, a software engineer doesn't need to have profound expertise in machine learning but has to understand the vocabulary of their fellow data analysts or scientists. According to the other culture, all engineers in a machine learning team must have a combination of machine learning and software engineering skills.

Besides having machine learning and software engineering skills, a machine learning team may include experts in data labeling and data engineering experts. DevOps engineers work closely with machine learning engineers to automate model deployment, loading, monitoring, and occasional or regular model maintenance.

A machine learning projects can fail for many reasons, and most actually do. Typical reasons for a failure are:

- lack of experienced talent,
- lack of support by the leadership,
- missing data infrastructure,
- data labeling challenge,
- siloed organizations and lack of collaboration,
- technically infeasible projects, and
- lack of alignment between technical and business teams.